

CIRCULAÇÃO LITERÁRIA INDEPENDENTE

FOLHEIO

Arquitetura, implementação
e operação

Guia do site, da API compartilhada
e do aplicativo Android em Kotlin

Edição	1.0
Data	08 de outubro de 2026
Código analisado	f96a04a
Branch funcional	codex/integracao-api-web-android
Publicação deste guia	main

Este documento descreve o código existente, como executar o sistema e como concluir a entrega. Funcionalidades futuras são identificadas como planejadas. Compilar o aplicativo não equivale a validar seu uso.

Sumário

1	Objetivo, escopo e versão	2
1.1	Código de referência	2
1.2	Primeira entrega	2
2	Estado real e evidências	2
3	Arquitetura e responsabilidades	3
3.1	Mapa do código	3
3.2	Tecnologias verificadas	4
4	Modelo de dados e regras	4
4.1	Limites de validação	4
5	Contrato de comunicação	5
5.1	Envelope e sessão	5
5.2	Rotas implementadas	5
5.3	Exemplos	5
6	Como executar	6
6.1	Checkout e pré-requisitos	6
6.2	API e site	6
6.3	URLs e configuração	7
6.4	Android Studio, SDK e Java	7
7	Validação funcional e visual	7
7.1	Roteiro cruzado	7
7.2	Revisão visual	8
8	Testes reproduzíveis	8
8.1	Go	8
8.2	Playwright	8
8.3	Aprovação da entrega	9
9	Como desenvolver e contribuir	9
9.1	Sequência de implementação	9
9.2	Migrações e organização	9
9.3	Git	9
10	Etapas seguintes	10
11	Diagnóstico e manutenção	10

1 Objetivo, escopo e versão

O FOLHEIO conecta leitores para anúncio, descoberta e negociação de livros por troca, venda e doação. Site e aplicativo consomem a mesma API e banco. A conta pode ser usada nos dois clientes, que mantêm sessões independentes. Este guia atende quem vai executar a demonstração, desenvolver funcionalidades e aprovar a entrega. A execução está na Seção 6; os critérios de aprovação, na Seção 7.

1.1 Código de referência

O código documentado está no commit `f96a04a`, na branch `codex/integracao-api-web-android`. Publicar este guia na `main` não incorpora automaticamente aquela implementação. Os comandos deste guia exigem checkout da integração ou de uma versão que a tenha incorporado. A especificação de máquina está em `backend/contrato.openapi.json`; o resumo em `CONTRATO_API.md`, na integração. Contrato, handlers e clientes devem ser alterados juntos.

1.2 Primeira entrega

- Cadastro, login, expiração e revogação da sessão.
- Catálogo público, pesquisa, publicação, edição e remoção de anúncios.
- Troca, venda e doação com capa opcional; perfil com estante do titular.
- Conversas por livro e mensagens persistidas, restritas aos participantes.
- SPA Vue e aplicativo nativo Kotlin conectados ao contrato compartilhado.

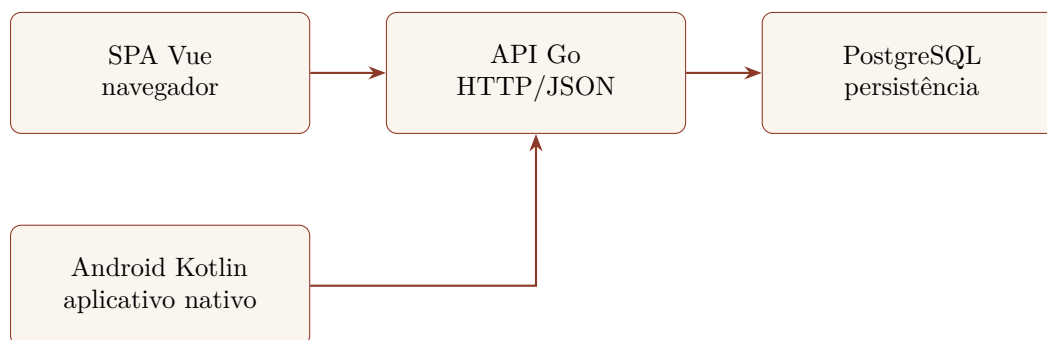
Venda representa anúncio com preço e negociação pelo chat. Não há pagamento integrado, frete nem registro formal de transação concluída. Avaliações do widget React pertencem ao protótipo; não integram a persistência da API desta entrega.

2 Estado real e evidências

Área	Evidência	Limite
API	Testes de regras e fluxo com PostgreSQL isolado executados durante a implementação.	Sem ensaio de carga ou auditoria de produção.
SPA	Build e teste de dois usuários: cadastro, anúncio, chat, edição, remoção e saída.	Responsividade e visual precisam de revisão adicional.
Android	Build debug concluído e APK gerado.	Execução em dispositivo e visual pendentes.
Python	Serviço inicial com recomendações fixas.	Sem recomendador conectado aos dados reais.
Mongo/Redis	Definidos no Compose.	Sem uso no fluxo integrado.

As evidências são pontuais. Um teste Go com `SKIP` por ausência de `TEST_DATABASE_URL` não valida o banco. PUT foi exercitado no teste web; ampliar a cobertura direta da edição no teste da API permanece uma tarefa.

3 Arquitetura e responsabilidades



A API autentica, valida regras, autoriza operações e grava no banco. Os clientes apresentam os dados e enviam intenções. Propriedade e participação em conversas são decididas pelo servidor.

3.1 Mapa do código

Caminho	Responsabilidade
<code>backend/api-core/cmd/server/main.go</code>	Inicialização HTTP, banco e CORS.
<code>backend/api-core/internal/plataforma/servidor.go</code>	Handlers, autenticação, regras e persistência.
<code>backend/api-core/internal/plataforma/esquema.sql</code>	Tabelas e índices iniciais.
<code>backend/api-core/internal/plataforma/servidor_test.go</code>	Testes de regras e integração.
<code>backend/contrato.openapi.json</code>	OpenAPI 3.0.3.
<code>folheio-vue/src/servicos/api.js</code>	Cliente HTTP, sessão e timeout.
<code>folheio-vue/src/paginas/</code>	Telas do site integradas.
<code>folheio-android/app/src/main/java/com/felipe/folheio_app/MainActivity.kt</code>	Telas e navegação programáticas.
<code>folheio-android/app/src/main/java/com/felipe/folheio_app/dados/ClienteApi.kt</code>	Cliente HTTP/JSON Android.
<code>backend/ai-service/app/main.py</code>	Serviço Python inicial.
<code>testes/integracao_web.cjs</code>	Teste Playwright de dois usuários.
<code>paginas/, scripts/</code>	Protótipo HTML original.
<code>folheio-react-widget/</code>	Widget de avaliações do protótipo.

3.2 Tecnologias verificadas

SPA: Vue 3.5, Vue Router 4.6 e Vite 8.3. Widget: React 19. API: Go, database/sql, pgx e bcrypt. Compose: PostgreSQL 15, Redis 7 e MongoDB 6. Android: AGP 9.4.0, Gradle Wrapper 9.6.0, SDK de compilação/alvo 37 e API mínima 24. O build foi executado com JBR 25 do Android Studio. Consulte manifests, lockfiles e catálogo Gradle antes de atualizar dependências.

4 Modelo de dados e regras

Entidade	Conteúdo e relações
Usuário	Identificador, nome, e-mail único, hash de senha, cidade e criação.
Sessão	Hash do token, usuário e expiração.
Livro	Proprietário, título, autor, gênero, conservação, descrição, modalidade, desejo, preço em centavos, capa e criação.
Conversa	Livro e interessado; combinação livro/interessado única.
Mensagem	Conversa, remetente, texto e criação.

O proprietário do livro e o interessado participam da conversa. Solicitações repetidas recuperam a conversa existente. Não se permite negociar o próprio livro. Excluir um anúncio exclui suas conversas e mensagens por **ON DELETE CASCADE**. A confirmação na interface deve comunicar esse efeito. Arquivamento e preservação do histórico ainda não existem.

4.1 Limites de validação

Texto é limitado em bytes no Go; caracteres acentuados podem ocupar mais de um byte. O cliente deve apresentar a mensagem de validação da API.

Campo	Regra
Título/autor	Obrigatórios; até 200 bytes cada.
Gênero	Obrigatório; até 80 bytes.
Descrição	Obrigatória; até 3.000 bytes.
Desejo	Até 500 bytes; obrigatório para troca.
Conservação	novo, seminovo, bom, marcas.
Modalidade	troca, venda, doacao.
Preço	Inteiro em centavos; venda positiva até 100.000.000; outras modalidades usam zero.
Capa	Opcional: JPEG/PNG Base64 em data URL, até 5 MiB de imagem; bytes validados.
Mensagem	Texto não vazio, até 2.000 bytes.
Cadastro	Nome/cidade obrigatórios até 120 bytes; e-mail válido até 254 bytes; senha entre 8 e 72 bytes.

O corpo JSON tem limite de 8 MiB. Base64 aumenta o tamanho da imagem; limite de corpo e de imagem são diferentes. Capas ficam em texto no banco nesta etapa; armazenamento de objetos é uma evolução prevista.

5 Contrato de comunicação

5.1 Envelope e sessão

```
{ "status": "sucesso", "code": 200, "data": {} }
```

Erros usam `status: erro` e `data.message`. Observe também o status HTTP. Cadastro/login retornam `token`, `expira_em` e `usuario` dentro de `data`. Rotas privadas recebem:

```
Authorization: Bearer TOKEN
Content-Type: application/json
```

O token opaco tem 256 bits aleatórios, expira em 24 horas e seu hash SHA-256 fica no banco. A saída revoga a sessão atual. Senhas usam `bcrypt`; JWT não foi implementado. A SPA mantém sessão em `sessionStorage`; o Android em memória, exigindo login ao reiniciar o processo. Livros e mensagens pertencem ao backend. Resposta 401 deve limpar a sessão e orientar novo login.

5.2 Rotas implementadas

Método	Caminho	Acesso/efeito
GET	/status	Público; API e banco.
POST	/api/v1/auth/cadastro	Público; conta/sessão.
POST	/api/v1/auth/login	Público; sessão.
POST	/api/v1/auth/sair	Autenticado; revogação.
GET	/api/v1/perfil	Titular e seus livros.
GET	/api/v1/livros	Público; catálogo/filtros.
POST	/api/v1/livros	Autenticado; anúncio.
PUT	/api/v1/livros/{livro}	Proprietário; edição.
DELETE	/api/v1/livros/{livro}	Proprietário; remoção.
GET	/api/v1/conversas	Participante; lista.
POST	/api/v1/conversas	Cria/recupera por livro.
GET	/api/v1/conversas/{conversa}/mensagens	Participante; leitura.
POST	/api/v1/conversas/{conversa}/mensagens	Participante; envio.

O catálogo aceita `busca` e `modalidade`; pesquisa título, autor, gênero e cidade, retornando até 100 anúncios recentes. Mensagens retornam até 200 recentes em ordem cronológica. Ainda não há paginação.

5.3 Exemplos

Cadastro:

```
{ "nome": "Leitora de demonstracao",
  "email": "leitora@example.com",
  "senha": "DEFINA_UMA_SENHA_DE_TESTE",
  "cidade": "Maceio - AL" }
```

Publicação/edição:

```
{ "titulo": "Duna", "autor": "Frank Herbert",  
  "genero": "fantasia", "estado": "bom",  
  "descricao": "Edicao conservada, sem anotacoes.",  
  "modalidade": "troca", "desejo": "Outro romance",  
  "preco_centavos": 0, "capa": "" }
```

Criar conversa e enviar mensagem são requisições distintas:

```
{ "livro_id": "IDENTIFICADOR_DO_LIVRO" }  
{ "texto": "Tenho interesse na troca." }
```

Use os identificadores retornados pela API. O proprietário vem do token; nomes e cidades apresentados vêm dos dados persistentes.

6 Como executar

6.1 Checkout e pré-requisitos

Em um clone novo:

```
git clone https://github.com/brianfigueredo0209-alt/folheio.projeto.git  
cd folheio.projeto  
git switch codex/integracao-api-web-android
```

Preserve alterações locais antes de trocar de branch. O Android versionado está em `folheio-android/`; cópias em `AndroidStudioProjects` não são sincronizadas automaticamente. Use Docker/Compose e Node/npm compatíveis com dependências/lockfile. O fluxo Docker não exige Go no host.

6.2 API e site

Na raiz:

```
docker compose up -d --build api-core  
docker compose ps  
Invoke-WebRequest http://localhost:8080/status  
npm ci --prefix folheio-vue  
npm run dev --prefix folheio-vue
```

Vite fica ativo no terminal. Abra <http://127.0.0.1:5173/>. Compose inicia o PostgreSQL, aguarda sua saúde e inicia a API. Redis, MongoDB e Python não são necessários ao fluxo inicial. Credenciais do Compose são de desenvolvimento. Volumes preservam dados. `docker compose down` encerra serviços; acrescentar `-v` remove volumes e dados, portanto não é solução rotineira de conexão.

```
docker compose logs --tail 100 api-core postgres
```

Confira Docker, portas, banco e logs antes de alterar código. Não publique tokens/senhas em relatos de erro.

6.3 URLs e configuração

Cliente	API
Navegador no computador	<code>http://localhost:8080</code>
Emulador Android	<code>http://10.0.2.2:8080</code>
Celular na mesma rede	<code>http://IP_DO_COMPUTADOR:8080</code>
Distribuição	Endpoint HTTPS do backend publicado.

`localhost` no celular é o próprio celular. O emulador usa `10.0.2.2` para alcançar o host; o celular físico usa IP do computador e precisa alcançar a porta. A SPA lê `VITE_API_URL`; a API lê `DATABASE_URL`, `PORT`, `CORS_ORIGINS`. CORS exige protocolo, host e porta exatos da origem, sem caminho. O Compose permite `localhost` e `127.0.0.1` na porta 5173. CORS não substitui autorização.

6.4 Android Studio, SDK e Java

Abra `folheio-android/`, configure SDK/JDK do Gradle e espere o Sync. Referência: JBR 25, plataforma 37 e Build Tools 36/37. `local.properties` é local e não deve entrar no Git. No Device Manager, crie uma imagem compatível com API mínima 24, inicie o dispositivo e selecione-o. *No Devices* indica ausência de dispositivo disponível. Na pasta Android:

```
.\gradlew.bat :app:assembleDebug
```

APK: `app/build/outputs/apk/debug/app-debug.apk`. Para celular:

```
.\gradlew.bat :app:assembleDebug -PfolheioApiUrl=http://IP_DO_COMPUTADOR:8080
```

Execute a configuração `app` com o backend ativo. HTTP está permitido apenas no manifesto debug; distribuição exige HTTPS. **Preview XML.** `activity_main.xml` ainda mostra Hello World. As telas funcionais são montadas em `MainActivity.kt`. O preview XML não representa a UI real: avalie o aplicativo instalado.

7 Validação funcional e visual

7.1 Roteiro cruzado

Crie contas distintas A e B. Não há credencial universal definida. Contas de `folheio_teste` não existem automaticamente na demonstração.

1. A publica troca com capa no site.
2. B entra no Android, atualiza catálogo e encontra a oferta.
3. B inicia conversa; A responde no site; B verifica a resposta.
4. A edita título, muda para venda e informa preço; B confere após atualizar.
5. A remove após ler a confirmação; confira remoção do anúncio e conversas.
6. Saia da sessão; rotas privadas devem pedir login.

O chat consulta a cada cinco segundos enquanto ativo. WebSocket e notificações de segundo plano não foram implementados. Teste senha incorreta, e-mail duplicado, rede indisponível, expiração, capa inválida, troca sem desejo e venda sem preço. Por requisição direta, B não deve editar anúncio de A; terceiro usuário não deve ler a conversa.

7.2 Revisão visual

Android precisa de teste em dispositivo. O fluxo web foi exercitado em desktop; isso não aprova automaticamente responsividade.

Tela/estado	Critério
Login/cadastro	Campos legíveis, erros úteis, teclado adequado e botão alcançável.
Catálogo	Capas proporcionais, fallback, busca e modalidade reconhecível pelo texto.
Publicação/edição	Rolagem completa; carga inicial não sobrescreve edição; preço coerente.
Perfil	Estante do titular; ações claras; confirmação explica efeito sobre chat.
Mensagens	Remetente reconhecível, texto longo sem corte, envio com teclado aberto.
Erro/vazio/carga	Recuperação possível; nenhum conteúdo fictício apresentado como persistente.
Acessibilidade	Texto ampliado, contraste, foco e ações compreensíveis ao leitor de tela.

Revise telas pequenas/grandes, textos longos, falta de capa e rede lenta. Registre captura, dispositivo, dimensões e reprodução. Captura estática não substitui interação.

8 Testes reproduzíveis

8.1 Go

Na pasta `backend/api-core/`:

```
go test ./...
```

Para integração, prepare PostgreSQL separado com banco `folheio_teste`:

```
$env:TEST_DATABASE_URL = "postgres://USUARIO:SENHA@HOST:5432/folheio_teste?sslmode=disable"
go test ./... -count=1 -v
```

A checagem de nome na URL não garante isolamento. Confira host, banco e credenciais; não use produção/demonstração. O teste Go tenta limpar seus usuários no final.

8.2 Playwright

`testes/integracao_web.cjs` exige Chromium/Playwright, site em 127.0.0.1:5173 e API de teste em localhost:8081 com banco isolado. Redireciona chamadas originalmente destinadas a localhost:8080. Prepare API separada: PORT=8081, DATABASE_URL de teste e CORS do site; execute Go em outro terminal, ou publique 8081 para 8080 em contêiner separado. A demonstração pode continuar em 8080. Na raiz, ferramentas locais:

```
npm install --prefix .ferramentas-teste playwright
.\.ferramentas-teste\node_modules\.bin\playwright.cmd install chromium
$env:PLAYWRIGHT_MODULE = (Resolve-Path .ferramentas-teste/node_modules/playwright).Path
node testes/integracao_web.cjs
```

Não versiona `.ferramentas-teste`. O teste cria contas no banco isolado e captura em `tmp/estante-integrada.png`. Não limpa todas as contas; planeje limpeza separada.

8.3 Aprovação da entrega

1. API inicia com banco e passa testes pertinentes.
2. SPA compila e passa fluxo de dois usuários sem erros JavaScript.
3. Android gera APK e passa roteiro em dispositivo.
4. Site e app passam visual e estados de erro.
5. Contrato/instruções correspondem ao código.
6. Revisão resolve falhas conhecidas antes de incorporar na main.

Execução e visual do Android permanecem pendentes na referência deste guia.

9 Como desenvolver e contribuir

9.1 Sequência de implementação

Defina caso de uso e resultado observável. Especifique entrada, saída, autorização, erros e limites. Implemente persistência/handler, teste regras e integre os dois clientes. Finalize com documentação e evidência de uso. Mudanças compartilhadas exigem revisar `api.js` e `ClienteApi.kt`. Preserve campos em mudanças compatíveis; planeje transição/versionamento para alterações incompatíveis.

9.2 Migrações e organização

`CREATE TABLE IF NOT EXISTS` não transforma tabelas existentes. Novas colunas/restrições exigem migrações explícitas e análise dos dados. Remover volumes não demonstra migração. Extraia responsabilidades do handler e da Activity à medida que o sistema crescer, preservando contrato e testes. Reorganização de arquivos sozinha não entrega funcionalidade.

9.3 Git

```
git fetch origin
git switch -c codex/nome-da-mudanca origin/main
# implementar, verificar e documentar
git add CAMINHOS_REVISADOS
git diff --cached --check
git commit -m "feat: descreve o comportamento entregue"
git push -u origin codex/nome-da-mudanca
```

Selecione arquivos; exclua SDK, configurações locais, tokens, caches, builds e APKs. A revisão descreve problema, resultado e testes. Publicar este guia na main não incorpora código da integração.

10 Etapas seguintes

Etapa	Entrega	Conclusão
1	Demonstrar	Dispositivo executando, dados compartilhados e visual revisado.
2	Cobrir limites	PUT direto, sessão, permissões, rede e upload exercitados.
3	Escalar	Paginação, índices medidos e capas fora do JSON principal.
4	Contas	Recuperação de senha e edição de perfil nos clientes.
5	Negociação	Estados, conclusão e preservação de histórico definidos.
6	Mensagens	Necessidade de WebSocket/notificações definida e testada.
7	Publicar	HTTPS, configuração externa, backup/restauração, observabilidade e release.

Pagamentos, frete e recomendações personalizadas dependem de requisito de produto. MongoDB/-Redis não precisam entrar no fluxo só por existirem no Compose. Antes da publicação, repita os roteiros com API definitiva e APK release.

11 Diagnóstico e manutenção

SDK não especificado: confira plataforma instalada, caminho de SDK e JDK; sincronize novamente.

No Devices: inicie emulador ou conecte celular autorizado para depuração.

Conexão: confira API, URL do build e rede; emulador usa 10.0.2.2, celular usa IP do host.

CORS: compare a origem exata com CORS_ORIGINS.

Conta de teste: confira banco; cadastre no ambiente da demonstração.

Hello World: execute o app, cuja interface vem do Kotlin.

Chat demora: confira ciclo de cinco segundos, tela ativa e API.

Fonte: docs/FOLHEIO_DOCUMENTACAO.tex. Documento independente de imagens e arquivos adicionais. Compile no editor LaTeX do Codex ou com distribuição TeX e os pacotes declarados:

```
pdflatex -interaction=nonstopmode FOLHEIO_DOCUMENTACAO.tex
pdflatex -interaction=nonstopmode FOLHEIO_DOCUMENTACAO.tex
```

Duas passagens atualizam sumário/referências. Confira diagnósticos e PDF, especialmente tabelas e comandos. Não versiona auxiliares. Atualize data, commit, evidências e pendências a cada entrega. Este guia foi elaborado a partir do código local. Não declara aprovação visual do Android, implantação em produção ou funções planejadas como concluídas.